BRIDGE: Robust Prompt Refinement for Inter-Agent Communication in LLM-based Multi-Agent Systems

Weiqiang Jin[1][0000−0002−6656−6061], Tao Tao[2], Shixiang Tang[1], Yang Liu[1], and Biao Zhao[1][0000−0002−3651−0702](✉)

[1] School of Information and Communications Engineering, Xi'an Jiaotong University, Xi'an, China
{weiqiangjin,shixiangtang,liu0725}@stu.xjtu.edu.cn, biaozhao@xjtu.edu.cn
[2] School of Computer Science and Technology, Anhui University of Technology, Ma'anshan, China
taotao@ahut.edu.cn

Abstract. Existing test-time prompt optimizers for LLM-based multi-agent systems, especially hierarchical reflection-style systems, mainly maximize reward on a small calibration set. In practice, that is not enough: a pipeline can work on the calibration examples and still break on paraphrases or mild peer-style changes. We call this failure mode *interface fragility*. We present BRIDGE, which turns each calibration example into a small set of stress tests for inter-agent communication. We use these stress tests to find where the handoff breaks, then keep a local repair only if it helps on held-out cases. We evaluate BRIDGE in the hierarchical reflect setting on six reasoning and coding benchmarks under two worker-evaluator configurations, including a transparent rule-based gate and a Bayesian repair-gate variant. Both the rule-based BRIDGE and its Bayesian repair-gate variant deliver substantial and consistent performance gains over Hierarchical MAS + MASPO across both the Qwen3-8B/Gemini-2.5-pro and the matched DeepSeek-v4-flash/DeepSeek-v4-pro settings. The diagnostics show that peer-style and rewrite probes mainly serve as stress tests for identifying brittle handoffs. These results suggest that calibration data is most useful when it exposes and repairs brittle interfaces rather than serving only as a scalar reward source.

Keywords: Multi-agent systems · Prompt optimization · Robustness · Interface repair · Collaboration diagnostics

1 Introduction

Large language model based multi-agent systems (MAS) are increasingly used for mathematical reasoning, science question answering, and code generation, where multiple specialized agents exchange intermediate messages before a final answer is produced [1,2]. In such pipelines, a system prompt does more than

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2027

define a local role. It affects what evidence gets passed on, how the message is formatted, and whether downstream agents can read it reliably. Prompt design in MAS is therefore partly an interface-design problem: a prompt that looks strong in isolation can still destabilize the pipeline if the message it produces is brittle or overly tailored to a particular collaborator.

Recent work on multi-agent prompt optimization, especially in hierarchical reflection-style systems, mainly optimizes prompts on a small calibration set. Methods such as MASPO [3] and MAPRO [4] differ in search strategy and credit decomposition, but they share the same basic setup: calibration examples provide a compact reward signal, and prompt updates are chosen by how well they score on that signal. This has produced meaningful gains, but it does not directly test whether the resulting interface stays stable when the same task is phrased differently or when a downstream peer uses a slightly different style.

This omission matters because prompt optimization in MAS can overfit not only to answers, but also to communication habits. A planner may learn to omit constraints that only one familiar verifier can reconstruct. A reflector may rely on formatting cues that disappear under paraphrase. A downstream judge may be blamed for a failure that was already baked into an upstream message. The main problem is *interface fragility*: the agents work well on the calibration examples, but the same handoff breaks under paraphrases or small changes in peer style.

We address this issue with BRIDGE. Instead of using each calibration item only once as a reward event, we turn it into a small set of stress tests for the collaboration pipeline. We first audit the system across these stress tests, then localize the likely failure point, and finally apply a local repair only if it improves held-out robustness without hurting clean performance too much.

Empirically, reward-driven baselines show clear interface fragility under held-out rewrites and peer-style perturbations. Across two worker-evaluator configurations, BRIDGE improves end-task performance, narrows held-out rewrite gaps on representative datasets, and uses peer-style perturbations mainly as diagnostic stress tests that help us see what was repaired.

Our main contributions are:

- We identify *interface fragility* as a key bottleneck in reward-driven MAS prompt optimization, and we test it with controlled rewrites and peer-style perturbations.
- We present BRIDGE, which stress-tests collaboration, localizes brittle handoffs with message replacement, and repairs them with local gated updates.
- We show on six reasoning and coding benchmarks that this view improves end-task performance, reduces held-out rewrite gaps, and yields useful robustness diagnostics in the hierarchical reflect setting studied here.

2 Related Work

Research on LLM-based MAS has explored both collaboration structure and prompt optimization. Early frameworks such as AutoGen and multi-agent de-

bate demonstrated that interaction among specialized agents can improve reasoning quality and controllability [1,2]. Later work, including MASPO [3] and MAPRO [4], turned prompt and topology design into an explicit optimization problem. BRIDGE is closest to this line, but differs in what it treats as the optimization signal: not only calibration reward, but evidence that collaboration remains stable under controlled perturbations.

Prompt design more broadly has been studied in single-agent reasoning and iterative revision settings. CoT [5], self-consistency [6], and Self-Refine [7] show that prompting, sampling, and revision can materially change system behavior. These methods provide useful design intuition, but they do not explicitly target the message interfaces that arise when multiple agents consume one another's intermediate outputs.

BRIDGE is also related to behavioral robustness testing and intervention-based analysis. Metamorphic testing, CheckList, and counterfactually augmented data all emphasize testing systems through controlled perturbations rather than through aggregate accuracy alone [8]. Meanwhile, intervention-oriented studies in neural models have shown the value of replacing or editing internal states to probe functional responsibility [9]. Our method brings these ideas together in a multi-agent prompt-optimization setting: it uses perturbation contexts to surface collaboration failures and message replacement interventions to decide where a local repair should be attempted.

3 Method

At a high level, BRIDGE does four things. It expands each calibration item into stress-test contexts, audits the resulting traces, localizes failures with message replacement, and applies a local repair that must pass a held-out gate. We use *stress-test contexts* as the umbrella term for both rewrite contexts and peer-style contexts, and *interface stability* for the property that collaboration remains reliable across them.

3.1 Prompt Optimization as Interface Stabilization

We model a multi-agent system as a directed acyclic graph $G = (V, E)$ of strategic agents. Each agent $v_i \in V$ is associated with a system prompt p_i and an LLM rollout function f_i. Given an input question q and the concatenated messages from its in-neighbors C_i, agent i produces $o_i = f_i(p_i, q, C_i)$, and the final system output is $o_{\text{glob}} = \Phi(G, P, q)$ for prompt set $P = \{p_i\}_{i=1}^{|V|}$.

Existing prompt optimizers update P to maximize a reward measured on a small calibration pool $U = \{q_1, \ldots, q_m\}$. BRIDGE keeps the same optimization setting but changes the target. We do not treat prompts only as local instructions. We treat them as interfaces that downstream agents must read reliably.

3.2 Stress-Test Contexts and Audit Criteria

For each calibration question q, BRIDGE constructs a small bundle of collaboration contexts $\mathcal{S}(q) = \mathcal{S}_{\text{input}}(q) \cup \mathcal{S}_{\text{peer}}(q)$. An *input context* preserves the answer

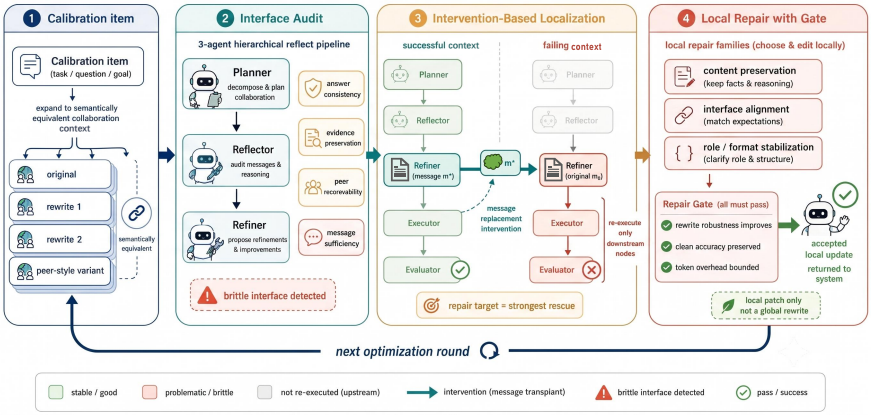


Fig. 1. Overview of BRIDGE. A calibration question is expanded into collaboration stress tests, checked for interface stability, traced by substituting messages across paired runs, and repaired through a local prompt or interface update.

while presenting the question in a new surface form. In a *peer context*, the task text is unchanged, but one downstream role is served by an alternative prompt with a different style and the same responsibility. In the main experiments, each question receives three input contexts and one peer context.

These contexts are not intended to simulate arbitrary distribution shift. They are targeted stress tests for a narrower property: whether the collaboration pipeline remains stable under perturbations that should not invalidate the task itself. Input contexts probe robustness to surface-form changes, while peer contexts probe whether agents have overfit to a private communication style instead of producing generally consumable intermediate messages.

The audit asks whether the MAS behaves compatibly throughout this context bundle. We track four invariants, but only I1 and I3 enter the main optimization score; I2 and I4 are used mainly to guide repair selection:

```
I1: Final-output consistency. The answer produced for the seed question
    should match the answers obtained from its semantic rewrites.
I2: Evidence preservation. Decision-relevant evidence in an upstream message
    should persist across equivalent contexts.
I3: Peer recoverability. Substituting an equivalent downstream peer should
    leave the system-level answer unchanged.
I4: Intermediate sufficiency. The downstream consumer should be able to draw
    a stable conclusion from the upstream handoff alone.
```

The repository also contains an uncertainty-calibration probe; we omit that probe from the objective reported here so that BRIDGE and MASPO use comparable optimization budgets. In practice, I1 checks final answers across the original and rewritten questions. I2 checks whether key constraints, numbers, or exceptions disappear from intermediate messages. I3 checks whether the same collaboration survives a downstream style swap. I4 checks whether an upstream message contains enough grounded information for the next agent.

Table 1. Main-text view of the repair policy in BRIDGE. These three families cover the repair choices we use in the main text.

Repair family	Typical failure pattern	Update scope
Content preservation	Missing constraints, destructive compression	Local prompt contract
Interface alignment	Peer-style brittleness, downstream misread	Edge/interface contract
Role/format stabilization	Role drift, brittle formatting, poor hedging	Role or format policy

For a mini-batch $B_t \subset U$ at round t, we summarize the audit with the violation rate

$$V_t = \frac{1}{|B_t|} \sum_{q \in B_t} \mathbf{1}\{\text{at least one active invariant is violated in } \mathcal{S}(q)\}. \quad (1)$$

We use this violation rate as the outer-loop progress signal. In the current implementation, the held-out gate relies on rewrite consistency together with clean-context correctness, because rewrites are the most stable held-out probes in our setup. Peer-style contexts still matter, but mostly as a diagnostic trigger rather than the main acceptance criterion.

3.3 Intervention-Based Localization

When a question exhibits an interface failure, BRIDGE pairs a successful context c_s with a failing context c_f for the same base item. It then probes each strategic agent by substituting one message across the two traces. For agent i, the message $o_i^{(c_s)}$ from the successful run is inserted into the failing run, after which only the descendants of i are re-executed. We judge the downstream behavior with a three-level rescue signal: if the inserted message restores the final answer, the agent receives a positive rescue signal; if the final behavior is effectively unchanged, it receives a neutral signal; and if the substitution moves the output farther from the successful run, it receives a negative signal. The default repair target is the component with the strongest rescue effect. In practice, this judgment chooses either the prompt of the implicated agent or, when the failure is clearly about message handoff, the local edge/interface contract attached to that agent's outgoing message.

We use the term *intervention-based localization* rather than full causal identification. The question here is practical: which message replacement is most likely to rescue the failed run? That is enough to guide a local repair. In Section 5.3, we evaluate it as a repair-targeting tool rather than as a general causal claim.

3.4 Local Repair with a Held-Out Gate

Once we localize a failure, BRIDGE applies a local repair rather than a full prompt rewrite. In the main text, we group repairs into the three families in Table 1: preserving missing information, aligning an interface, and stabilizing role or format behavior. The goal is simple: keep the parts that already work and change only the brittle handoff.

Before a local edit is retained, BRIDGE tests it on $H_t \subset U \setminus B_t$, a held-out slice disjoint from the current mini-batch. We instantiate two acceptance gates for this decision; both restrict the edit to the localized component, but they estimate out-of-batch reliability in different ways. We use BRIDGE-H and BRIDGE-B as compact names for the heuristic and Bayesian variants.

BRIDGE (heuristic rule-based). The transparent version uses the original deterministic held-out gate. It compares the before/after tuple formed by robustness violations, clean-context correctness, and token overhead. A patch is accepted only if it improves held-out robustness without unacceptable regression on the clean context: $\text{ACCEPT}_{\text{H}} \iff \tilde{V}_{\text{held}} < V_{\text{held}} \ \wedge \ \tilde{C}_{\text{held}} \geq C_{\text{held}} - 1 \ \wedge \ \tau < 1.15$, where $\tilde{V}_{\text{held}}$ is the held-out robustness proxy after the patch, $\tilde{C}_{\text{held}}$ is held-out clean-context correctness, and τ is relative token overhead. This version is intentionally conservative and easy to inspect, so we use it as the default variant for diagnostics and ablations.

BRIDGE (Bayesian repair gate). The learned variant augments the local-repair mechanism with a lightweight Bayesian neural acceptor. For a candidate patch p, we build a feature vector z_p from held-out robustness deltas, clean-context deltas, localization confidence, repair family, prompt-length change, and probe mix. A pretrained calibration encoder followed by a Bayesian last-layer MLP estimates the posterior probability that the patch will remain beneficial on the next held-out audit:

$$P_{\text{keep}}(p) = \mathbb{E}_{\theta \sim q_\phi(\theta)} \left[\Pr(y_p = 1 \mid z_p, \theta) \right]. \tag{2}$$

The Bayesian gate accepts a patch when the posterior mass favors retention and the lower credible bound of the predicted repair gain remains positive. This keeps the same local-edit discipline as the heuristic gate, but lets uncertainty over similar past repairs decide when a patch is too brittle to keep.

Why local repair? This design is especially important in the small-calibration regime. Whole-prompt rewrites can improve one failure mode while silently disturbing instructions that were already working. Local repair constrains the update to the part of the interface most closely tied to the diagnosed failure, which can also yield interpretable diagnostics in addition to end-task gains.

4 Experimental Setup

Benchmarks. We use six tasks: MATH-500 [10], AGIEval-MATH [11], AQuA [12], GPQA-Diamond [13], MBPP [14], and HumanEval-ET [15]. Together, they cover quantitative reasoning, science question answering, and program synthesis/code evaluation. Scores are rounded percentages, $\text{round}(100\,\text{correct}/N, 2)$; Avg_6 averages the six task scores.

Backbones and topology. Experiments are run with two worker–evaluator stacks. In *Qwen+Ge- mini*, Qwen3-8B serves as the worker model and Gemini-2.5-pro serves as the evaluator. In *DeepSeek*, DeepSeek-v4-flash is paired with DeepSeek-v4-pro in the same worker/evaluator roles. All systems use a MASPO-style three-agent hierarchical-reflect layout at depth two.

Scope. Although the title speaks to LLM-based MAS more broadly, the empirical evidence in this paper is intentionally grounded in the hierarchical reflect family just described. We use that setting to test the paper's main claim: prompt refinement is stronger when calibration data is used to stabilize inter-agent interfaces rather than only to optimize clean-context reward.

Baselines and optimization budget. For the Qwen+Gemini stack, the comparison set includes Vanilla, CoT [5], SC(CoT) [6], Self-Refine [7], AgentDropout [16], Hierarchical MAS + SPO, and Hierarchical MAS + MASPO [3]. For DeepSeek, we report the comparable baseline subset up to and including MASPO. Unless otherwise noted, every method is calibrated on an identical set of 50 examples. Both BRIDGE variants start from that same 50-item pool and add stress-test rollouts plus a held-out repair gate. In the supplementary robustness analyses, we additionally instantiate an auxiliary role-pressure probe family that is supported by the framework but not part of the core main-table setting.

Variant used in follow-up analyses. Tables 2 and 3 report both BRIDGE-H and BRIDGE-B in the main accuracy comparison. Unless explicitly stated otherwise, the subsequent robustness, localization, diagnostic, and ablation experiments are conducted with BRIDGE-B. We focus on BRIDGE-B because its Bayesian repair-gate policy is more flexible when held-out probes provide heterogeneous evidence across tasks, while BRIDGE-H remains the transparent rule-based reference.

We focus on four questions. Does BRIDGE improve end-task performance? Does it improve held-out robustness? Does intervention-based localization find better repair targets? What kinds of failures does it repair most often?

5 Results and Analysis

Main findings. Across both worker-evaluator settings, the two BRIDGE variants improve end-task accuracy, and the diagnostics show that local repairs reduce collaboration violations while localizing failures more reliably under controlled injections. The supplementary analyses below also show that the margin grows under compound perturbations and that better target selection is a major part of why the accepted repairs stay useful.

5.1 Main Performance Results

Table 2 reports the cross-family Qwen3-8B/Gemini-2.5-pro setting. The heuristic rule-based BRIDGE outperforms Hierarchical MAS + MASPO on five of the six datasets and improves the six-task average by 3.15 points. The Bayesian repair-gate variant further improves the average, mainly by strengthening MATH-500, MBPP, and HumanEval-ET. The tradeoff is that the learned gate is less stable on GPQA-Diamond in this stack, which suggests that uncertainty-aware repair selection is useful but still sensitive to the calibration distribution.

Table 3 shows the matched DeepSeek-v4-flash/DeepSeek-v4-pro setting. Here the heuristic rule-based BRIDGE improves all six datasets and raises the six-task average by 6.25 points over Hierarchical MAS + MASPO. The Bayesian

Table 2. Qwen3-8B worker evaluated by Gemini-2.5-pro: task accuracy (%).

Method	MATH-500	AGIEval	AQuA	GPQA	MBPP	HumanEval-ET	Avg_6
Vanilla	74.80	55.86	79.53	45.96	63.47	71.95	65.26
CoT	75.40	54.69	81.89	46.72	64.17	72.26	65.86
SC(CoT)	75.50	56.64	82.17	47.52	64.49	72.56	66.48
Self-Refine	76.20	56.52	82.28	47.73	64.17	70.73	66.27
AgentDropout	76.80	59.77	86.23	47.98	58.09	65.86	65.79
Hier. MAS + SPO	77.80	63.41	86.61	51.01	61.83	73.39	69.01
Hier. MAS + MASPO	78.40	64.45	87.01	54.04	65.34	76.83	71.01
BRIDGE-H	85.80	**66.33**	**93.58**	**58.54**	61.91	78.78	74.16
BRIDGE-B	**88.10**	65.80	89.20	55.60	**67.50**	**93.58**	**76.63**

Table 3. DeepSeek-v4-flash worker paired with DeepSeek-v4-pro evaluator: task accuracy (%).

Method	MATH-500	AGIEval	AQuA	GPQA	MBPP	HumanEval-ET	Avg_6
Vanilla	55.60	38.67	76.38	32.32	60.42	80.49	57.31
CoT	56.20	39.45	78.35	33.33	60.66	81.10	58.18
SC(CoT)	56.80	41.02	79.13	34.34	60.89	81.71	58.98
Self-Refine	57.40	40.62	79.92	34.85	61.36	80.49	59.11
AgentDropout	57.80	42.97	81.50	36.36	56.21	73.78	58.10
Hier. MAS + SPO	57.90	42.58	81.89	35.86	62.77	83.54	60.76
Hier. MAS + MASPO	58.20	43.36	82.28	36.87	63.70	84.76	61.53
BRIDGE-H	69.80	54.30	84.65	**44.44**	**66.28**	**87.20**	67.78
BRIDGE-B	**72.10**	**55.80**	**86.90**	43.10	63.70	86.30	**67.98**

repair gate reaches a slightly higher average of 67.98, with gains concentrated on MATH-500, AGIEval, and AQuA. The pattern suggests that learned repair selection is most useful when its uncertainty estimates help choose among several plausible local edits.

5.2 Robustness Under Stress Tests

Figure 2 shows why stress-test calibration is necessary. Under the Qwen3-8B/Gemini-2.5-pro setting, reward-driven baselines show substantial drops under held-out rewrites and peer-style swaps, especially on GPQA-Diamond and HumanEval-ET. On GPQA-Diamond, held-out rewrites remove 6.12 points for MASPO and 5.47 for MAPRO, while peer-style swaps still cost 4.91 and 3.27 points. These drops are easy to miss in coarse benchmarking, but they are large enough to show that the inter-agent interface has overfit to a particular surface form or collaborator style.

Stress-family breakdown. As a supplementary diagnostic, Figure 3 breaks robustness down by probe family. For this supplementary audit, we use lexical rewrites, structural rewrites, peer-style swaps, and an auxiliary role-pressure family. Structural rewrites and peer-style swaps expose the largest weaknesses. On reasoning-heavy tasks, structural violations fall from 0.47 to 0.20 and peer-style violations from 0.36 to 0.16 after repair, while code-heavy tasks show a similar but slightly smaller pattern. Content-preservation repairs are most common for rewrite fail-

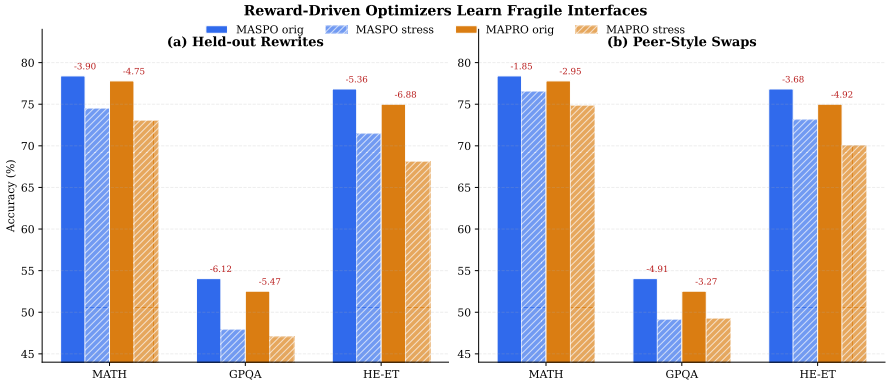


Fig. 2. Interface fragility of reward-driven baselines under the Qwen3-8B/Gemini-2.5-pro setting. Panel (a) shows accuracy degradation under held-out rewrites; panel (b) shows degradation under peer-style swaps. Both patterns motivate the stress-test view adopted by BRIDGE.

ures, while interface-alignment and role/format repairs appear more often under peer and role stress.

Low-budget regime. Test-time prompt optimization often has to work with very small calibration pools, so we also test whether BRIDGE helps before the full 50-example budget is available. Figure 4 varies the optimization pool over {10, 20, 30, 50} examples while keeping the gate fixed. Both stacks recover a useful part of their final robustness gain by 20 examples, and held-out violation reduction keeps improving even when clean accuracy starts to level off. For example, the Qwen+Gemini stack moves from 71.66 to 72.34 Avg_6 between 10 and 20 examples, but the held-out violation reduction already jumps from 17.9% to 26.4%. This matches the intended use of BRIDGE as a small-budget repair loop rather than a retraining-heavy method.

Probe-mix generalization. Table 4 asks whether robustness improves only on the probe family emphasized during optimization. The answer in this setting is encouraging but not absolute: adding peer and role stress generally reduces violations not only on seen rewrite probes, but also on unseen collaborator and role-pressure probes. The clearest movement appears on the collaborator-focused probes, where rewrite-only optimization leaves the largest residual failure surface. In the Q+G stack, unseen peer violations drop from 0.27 to 0.16 once the full probe mix is enabled, and the DeepSeek stack shows a similar reduction from 0.29 to 0.17.

Compound perturbations. Single-family probes are useful, but real failures often combine several mild shifts at once. We therefore build an additional compound-stress suite that composes rewrites, peer-style swaps, and role-pressure edits within the same held-out item. In an additional stress-only evaluation, Figure 5 shows a clear pattern: as the perturbation order increases, the accuracy gap between MASPO and BRIDGE widens rather than shrinks. In the Qwen+Gemini stack, MASPO drops from 72.4 under single stress to 64.7 under triple stress,

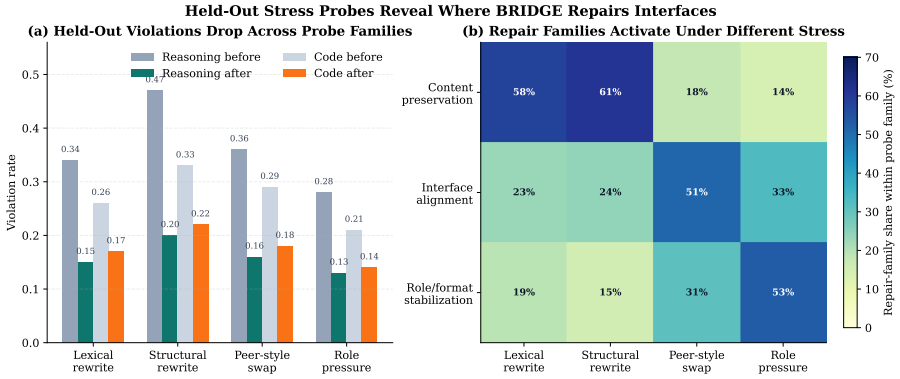


Fig. 3. Held-out stress-family analysis beyond the main accuracy tables. Panel (a) reports average violation rates before and after BRIDGE under four probe families for reasoning-heavy and code-heavy tasks. Panel (b) shows how accepted repairs distribute across repair families under the same probe families.

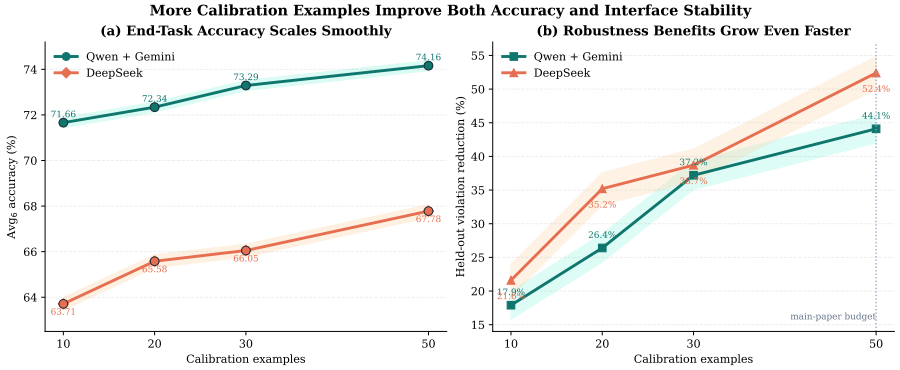


Fig. 4. Calibration-budget sensitivity under fixed optimization settings. Panel (a) shows Avg_6 as the calibration pool grows; panel (b) tracks the average held-out violation reduction across the four probe families used in the robustness audit.

while BRIDGE still stays at 70.6. The right panel shows the same trend at dataset level, with the largest margins appearing under rewrite+peer+role compositions on MATH-500 and GPQA-Diamond.

5.3 Localization Audit

If interface failures are to be repaired locally, the system must first localize where the collaboration breaks. Table 5 evaluates this step with a controlled injection audit. On AQuA, the injected error suppresses numerical constraints in the predictor; on MBPP, the reflector is forced to reject every plan. The injection site is therefore known, which makes blame@1 a meaningful metric. This audit isolates the diagnostic step before any prompt rewrite is attempted, so it gives a cleaner picture of what the localization signal itself is doing.

Table 4. Effect of optimization-time probe mix on held-out robustness. Each violation rate is measured on a held-out probe family not used for patch acceptance.

Cfg.	Probe mix	Seen rewrite viol. ↓	Unseen peer viol. ↓	Unseen role viol. ↓	Avg_6
Q+G	Rewrite only	0.19	0.27	0.31	72.84
Q+G	Rewrite + peer	0.20	0.19	0.27	73.41
Q+G	Full BRIDGE probes	0.15	0.16	0.19	74.16
DS	Rewrite only	0.21	0.29	0.33	65.94
DS	Rewrite + peer	0.18	0.20	0.29	66.72
DS	Full BRIDGE probes	0.14	0.17	0.21	67.78

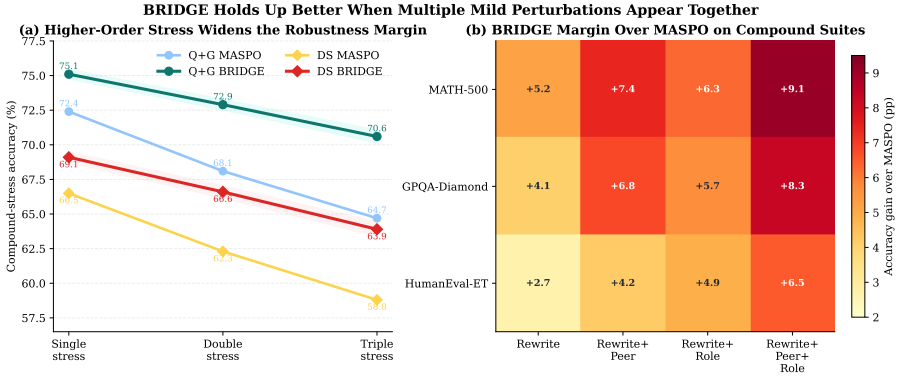


Fig. 5. Independent compound-stress evaluation beyond the main accuracy tables. Panel (a) compares average accuracy as one, two, or three perturbation families are combined within the same held-out item. Panel (b) reports the resulting BRIDGE gain over MASPO on representative compound suites.

The comparison is clear. Local reward attribution often points to a downstream stage that only inherits an upstream mistake, while intervention-based localization is much more likely to select the injected repair target in this controlled audit. On AQuA, blame@1 rises from 63.2 for MASPO local reward to 88.7 for BRIDGE, and the share of downstream false blame falls from 83.5% to 40.6%. We see the same pattern again on MBPP. In this setting, paired message replacement is a better repair-targeting tool than local reward decomposition.

Target-selection policy. The controlled injection audit tells us whether a method can identify the true source, but the optimizer still has to decide which stage to edit in practice. We therefore run an additional target-selection sweep with five policies: random choice, downstream-only edits, reward-targeted edits, the BRIDGE intervention target, and an oracle target that edits the true injected stage. Figure 6 shows that BRIDGE sits near the Pareto frontier in both stacks. In the Qwen+Gemini setting it repairs 63.7% of broken cases while retaining 96.8% clean performance, compared with 46.2% and 94.4% for reward-based targeting. The oracle remains better, so there is still headroom, but most of the practical gap is already closed without privileged labels.

Table 5. Controlled bug-injection audit for localization. Cells give blame@1 (%) for the injected source; parenthetical values are downstream-attribution errors. Two reward-only attribution baselines are shown for context.

Attribution method	AQuA (predictor bug)	MBPP (reflector bug)
MAPRO reward attribution	57.9 (86.4% downstream)	53.1 (79.8% downstream)
MASPO local reward	63.2 (83.5% downstream)	54.7 (78.6% downstream)
BRIDGE intervention	88.7 (40.6% downstream)	87.9 (32.8% downstream)

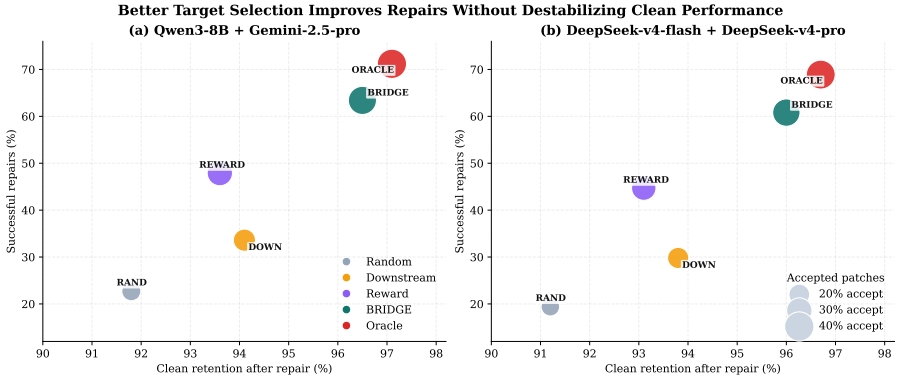


Fig. 6. Independent target-selection sweep for local repair. Each point is a stage-selection policy; the x-axis measures clean retention after repair, the y-axis measures successful repairs on a held-out broken set, and bubble size shows the share of accepted patches.

5.4 Repair Diagnostics and Ablation

Figure 7 shows the repair process more directly. Panel (a) tracks violation rates over optimization rounds, with the mean trajectory highlighted against the dataset-level range. Most of the drop happens in the first few accepted edits, after which the curve flattens into smaller refinements rather than oscillating. Panel (b) shows where accepted patches concentrate across datasets as a repair-class heatmap, with especially large shares on GPQA-Diamond and HumanEval-ET.

Table 6 isolates which ingredients are responsible for these repairs. Removing collaboration contexts hurts consistently, which confirms that robustness probes are not a cosmetic addition. The effect is strongest on MATH-500 and GPQA-Diamond, where the DeepSeek stack loses 5.60 and 5.55 points respectively. Removing intervention-based localization also degrades performance, especially on GPQA. Replacing local repair with broader rewrites hurts code-generation tasks the most, which is consistent with the view that narrowly scoped interface repairs are safer than global prompt churn when calibration data is limited.

Gate-policy comparison. The acceptance gate also matters. Table 7 compares clean-only, rewrite-only, and the joint heuristic rule in BRIDGE-H. The clean-only gate accepts too many brittle patches. The rewrite-only gate pushes rewrite robustness harder, but it also keeps noisier edits. The joint rule gives the best overall balance once we consider clean accuracy, robustness, and next-pass re-

Table 6. DeepSeek-v4-flash worker paired with DeepSeek-v4-pro evaluator: ablation results.

Variant	MATH-500	GPQA	MBPP	HumanEval-ET
Full BRIDGE	69.80	44.44	66.28	87.20
w/o Stress-Test Contexts	64.20 (5.60↓)	38.89 (5.55↓)	63.70 (2.58↓)	83.54 (3.66↓)
w/o Localization	66.40 (3.40↓)	40.40 (4.04↓)	65.10 (1.18↓)	85.37 (1.83↓)
w/o Local Repair	70.00 (0.20↑)	43.43 (1.01↓)	64.88 (1.40↓)	84.76 (2.44↓)

Table 7. Effect of the held-out acceptance gate. Rewrite gap is averaged over the three representative robustness datasets. Next-pass retention measures the share of accepted edits that remain beneficial after one additional held-out audit pass.

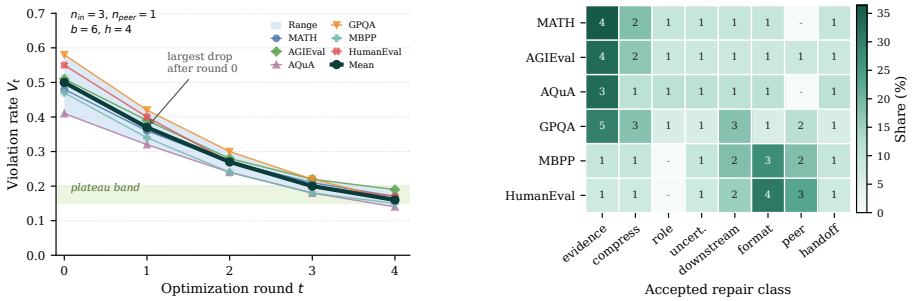
Cfg.	Gate policy	Avg_6	Rewrite gap ↓	Next-pass retention (%)
Q+G	Clean only	73.56	3.11	60.8
Q+G	Rewrite only	73.78	2.01	66.7
Q+G	Joint rule (BRIDGE-H)	74.16	2.09	72.8
DS	Clean only	66.92	2.68	63.4
DS	Rewrite only	67.36	1.71	68.7
DS	Joint rule (BRIDGE-H)	67.78	1.74	74.3

tention together. In the Q+G stack, for instance, next-pass retention rises from 60.8% under the clean-only gate to 72.8% under the joint rule, while Avg_6 also improves from 73.56 to 74.16.

MATH-500 case study. Table 8 shows two illustrative MATH-500 cases selected to demonstrate common repair patterns rather than to estimate aggregate frequency. In both, the original task is solved correctly, but a semantics-preserving rewrite changes how one key constraint is phrased. The reward-driven baseline then compresses away that condition, while BRIDGE keeps it explicit in the handoff. The two failures are also meaningfully different: Case A loses a modular constraint and drifts to the wrong candidate, whereas Case B drops an endpoint exclusion and corrupts the final solution set.

6 Conclusion

We presented BRIDGE, a prompt refinement method for LLM-based multi-agent systems that treats inter-agent communication as an optimization target rather than a byproduct of prompt tuning. Instead of using calibration data only as a reward signal, BRIDGE uses controlled stress tests to identify brittle interfaces, localize failures, and apply targeted repairs. Across six reasoning and coding benchmarks in the hierarchical reflect setting, BRIDGE consistently improves performance over strong reward-driven baselines while reducing sensitivity to held-out rewrites. These results suggest that a substantial portion of prompt-optimization failure arises not from weak individual agents, but from fragile communication between them. More broadly, our findings motivate viewing prompt optimization as an interface-stabilization problem. Extending this perspective to other multi-agent topologies and richer forms of collaboration remains an important direction for future work.



(a) Violation rate over optimization rounds.

(b) Accepted repair-class heatmap.

Fig. 7. Diagnostics for BRIDGE. The left panel tracks the decline in collaboration violations with a mean curve and dataset-level range; the right panel shows which repair classes are most frequently accepted, with cell values indicating accepted-patch counts.

Table 8. Two MATH-500 case studies illustrating interface repair under held-out rewrites. Message traces are shortened for readability but preserve the failures.

Case	Rewrite trigger	Reward-driven trace	BRIDGE repaired trace
MATH-500 Case A (number theory)	A remainder condition is restated as a **modular-congruence clause**, so the same arithmetic fact appears in a denser symbolic form.	The predictor first gives a faithful chain in the original context, but after rewriting it compresses the constraint list into "even near 100" and **drops the explicit congruence check**. The reflector then treats 96 as compatible with the shortened handoff because the missing modulus is no longer testable downstream. The refiner never recovers that omission, so the final answer flips only in the rewrite context even though the underlying math is unchanged.	**Content-preservation repair** asks the predictor to emit a short checklist rather than a free-form summary: parity, modulus, and range must all survive the handoff. The rewritten trace now **keeps both congruence conditions explicit**, so the reflector rejects 96 and the refiner returns 104 in both contexts. The repair is local, but it stabilizes the whole downstream chain.
MATH-500 Case B (piecewise algebra)	An **exception clause** is folded into longer prose in the rewrite, so the exclusion is no longer presented as a standalone condition.	The reflector compresses the branch analysis into a short union statement and **silently drops the forbidden endpoint**. That change looks minor because most of the interval structure is still correct, but it alters what the refiner believes is safe to include at the boundary. As a result, the rewrite context returns a set containing an invalid endpoint even though the original context solved the same problem correctly.	**Interface-alignment repair** tells the reflector to pass each branch in a stable slot-based schema: interval, inclusion rule, and exceptional endpoint. The refiner therefore **preserves the exclusion rather than paraphrasing it away**, and both contexts end with the same valid solution set. This case highlights that the repaired interface is not just more verbose; it is more consumable by the downstream agent.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

Acknowledgements. We are grateful to the editors and anonymous reviewers for their thorough and insightful comments, which have improved our work.

References

1. Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A.H., White, R.W., Burger, D., Wang, C.: Autogen: Enabling next-gen LLM applications via multi-agent conversations. In: First Conference on Language Modeling (2024)
2. Du, Y., Li, S., Torralba, A., Tenenbaum, J.B., Mordatch, I.: Improving factuality and reasoning in language models through multiagent debate. In: Proceedings of the 41st International Conference on Machine Learning (2024)

3. Wang, Z., Liu, X., Wang, L., Shan, Z., Wang, Y., Song, Z., Zhang, M.: MASPO: Joint prompt optimization for LLM-based multi-agent systems. In: Proceedings of the 43rd International Conference on Machine Learning (2026)
4. Zhang, Z., Ge, L., Li, H., Zhu, W., Zhang, C., Ye, Y.: MAPRO: Recasting multi-agent prompt optimization as maximum a posteriori inference. In: Findings of the Association for Computational Linguistics: EACL 2026. pp. 4458–4480 (2026)
5. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E.H., Le, Q.V., Zhou, D.: Chain-of-thought prompting elicits reasoning in large language models. In: Advances in Neural Information Processing Systems (NeurIPS) (2022)
6. Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E.H., Narang, S., Chowdhery, A., Zhou, D.: Self-consistency improves chain of thought reasoning in language models. In: The Eleventh ICLR (2023)
7. Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., et al.: Self-refine: Iterative refinement with self-feedback. In: Advances in Neural Information Processing Systems (NeurIPS) (2023)
8. Liang, L., Tan, C., Deng, Y., Cai, Y., Chen, T.Y., Zheng, X.: AutoMT: A multi-agent LLM framework for automated metamorphic testing of autonomous driving systems. arXiv preprint arXiv:2510.19438 (2025)
9. Geiger, A., Lu, H., Icard, T., Potts, C.: Causal abstractions of neural networks. In: Advances in Neural Information Processing Systems (NeurIPS) (2021)
10. Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., Steinhardt, J.: Measuring mathematical problem solving with the MATH dataset. In: Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (2021)
11. Zhong, W., Cui, R., Guo, Y., Liang, Y., Lu, S., Wang, Y., Saied, A., Chen, W., Duan, N.: AGIEval: A human-centric benchmark for evaluating foundation models. In: Findings of the Association for Computational Linguistics: NAACL 2024. pp. 2299–2314 (2024)
12. Ling, W., Yogatama, D., Dyer, C., Blunsom, P.: Program induction by rationale generation: Learning to solve and explain algebraic word problems. In: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 158–167 (2017)
13. Rein, D., Hou, B.L., Stickland, A.C., Petty, J., Pang, R.Y., Dirani, J., Michael, J., Bowman, S.R.: GPQA: A graduate-level google-proof Q&A benchmark. In: First Conference on Language Modeling (COLM) (2024)
14. Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., Sutton, C.: Program synthesis with large language models. arXiv preprint arXiv:2108.07732 (2021)
15. Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H.P., Kaplan, J., Edwards, H., et al.: Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 (2021)
16. Wang, Z., Wang, Y., Liu, X., Ding, L., Zhang, M., Liu, J., Zhang, M.: Agent-Dropout: Dynamic agent elimination for token-efficient and high-performance LLM-based multi-agent collaboration. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics. pp. 24013–24035 (2025)